

AI Automation with n8n

Lesson 5: Conditional Logic with AI Decisions

Conditional Logic with AI Decisions – Study Notes

Key Concepts

1. **If node** – a flow control element that evaluates a condition and routes the conversation down one of two (or more) branches.
2. **GPT (or any LLM) as a sentiment analyzer** – prompting the model to return a clear sentiment label (e.g., `positive`, `negative`, `neutral`).
3. **Connecting LLM output to the If node** – using the sentiment result as the variable the If node checks.
4. **Branch handling** – designing distinct responses or actions for each sentiment branch (e.g., empathy for negative, celebration for positive).
5. **Testing & debugging** – verifying that the LLM consistently returns the expected format and that the If node correctly interprets it.

Summary

Conditional logic lets you make your chatbot react differently depending on what the user says. In a beginner friendly setup, you combine an **If node** with a **GPT call** that evaluates the user's sentiment. First, the user's message is sent to GPT with a prompt that asks the model to classify the sentiment and return a simple, machine readable label (e.g., `"sentiment: positive"`). The flow then stores this label in a variable (often ``sentiment``).

Next, the **If node** reads the ``sentiment`` variable and compares it to the expected values. If the sentiment is `negative`, the flow follows the "negative" branch where you might apologize, offer help, or route to a human agent. If it's `positive`, the flow follows the "positive" branch, perhaps thanking the user or offering a promotion. A third "neutral/unknown" branch can catch any unexpected output. By chaining these two components—LLM sentiment analysis and the If node—you give the bot a basic decision making ability without writing complex code.

Important Points

- **Prompt design matters** – ask GPT to output only the sentiment label (e.g., ``sentiment: positive``) to avoid parsing errors.
- **Variable storage** – after the GPT call, store the result in a clearly named variable (``sentiment``) that the If node can access.
- **Exact matching** – the If node typically performs a string equality check, so ensure the label's spelling and case are consistent.
- **Fallback branch** – always include an "else" or "default" path for unrecognized sentiment values.
- **Performance tip** – limit the GPT request to a short prompt and a single token response; this reduces latency and cost.

- **Testing** – run a variety of user inputs (positive, negative, sarcastic, ambiguous) to confirm the end to end flow works reliably.

Examples

Example 1: Simple Positive/Negative Routing

1. **User message** ! "I'm really frustrated with the app crashing."
2. **GPT node prompt**:
```  
Classify the sentiment of the following text. Respond only with one word: positive, negative, or neutral.  
Text: "{{user\_message}}"  
```
3. **GPT response** saved to variable `sentiment` ! `negative`.
4. **If node** checks `sentiment == "negative"` ! **True** branch:
 - Bot replies: "I'm sorry you're experiencing issues. Let me connect you with support."
5. **Else** (positive/neutral) branch would send a different, upbeat reply.

Example 2: Adding a "Neutral" Path

1. **User message** ! "The update looks okay."
2. Same GPT prompt as above; GPT returns `neutral`.
3. **If node** logic:
 - **If** `sentiment == "positive"` ! thank the user.
 - **Else if** `sentiment == "negative"` ! apologize and help.
 - **Else** (covers `neutral` or any unexpected label) ! "Thanks for the feedback! Anything else I can help with?"

These examples illustrate how a single GPT call can drive multiple conditional branches, creating a more responsive conversational experience.

Quick Review

1. **What does an If node do in a chatbot flow?**
2. **Why should the GPT prompt ask for a single word sentiment label?**
3. **How do you store the GPT sentiment result for later use?**
4. **What is a good practice for handling unexpected sentiment values?**
5. **Name one way to reduce latency/cost when using GPT for sentiment analysis.**

Further Reading

- Prompt engineering basics – crafting reliable LLM prompts.
- Advanced conditional logic: using multiple If nodes and logical operators (AND/OR).
- Sentiment analysis techniques beyond LLMs (e.g., pretrained classifiers, VADER).
- Error handling and fallback strategies in conversational flows.
- Optimizing LLM usage: token budgeting, temperature settings, and response parsing.

